

# EE 210L — Digital Circuits and Logic Design Lab

## Lab 11: Counters and Memory on the FPGA

Name: \_\_\_\_\_ Date: \_\_\_\_\_

Partner(s): \_\_\_\_\_ Section: \_\_\_\_\_

*Do not start Part 2 until the LED in Part 1 is blinking. Everything below depends on a working clock. Submit this report on Canvas.*

### Part 1 — Verify the Clock

---

Divider width used: \_\_\_\_\_ bits

Time for ten blinks: \_\_\_\_\_ s    Period per blink: \_\_\_\_\_ s

Clock frequency computed from your measurement: \_\_\_\_\_ MHz    Expected: 125 MHz

Show your divider calculation:

---

---

### Part 2 — Counter on Hardware

---

Time for a full 0→15 cycle: \_\_\_\_\_ s    Predicted from your divider: \_\_\_\_\_ s

Does btn[0] reset the count? \_\_\_\_\_

| Resource   | Used | % Utilization |
|------------|------|---------------|
| LUTs       |      |               |
| Flip-flops |      |               |

### Part 3 — Ring Counter

---

Describe what the LEDs do, and how it differs from the binary counter:

---

---

Does this match the 74LS194 behavior from Lab 7? \_\_\_\_\_

**Part 4 — Button Without Debouncing**

---

| Trial | Presses made | Count reached | Extra counts |
|-------|--------------|---------------|--------------|
| 1     | 10           |               |              |
| 2     | 10           |               |              |
| 3     | 10           |               |              |
| 4     | 10           |               |              |

**Part 5 — Button With Debouncing**

---

| Trial | Presses made | Count reached | Extra counts |
|-------|--------------|---------------|--------------|
| 1     | 10           |               |              |
| 2     | 10           |               |              |
| 3     | 10           |               |              |
| 4     | 10           |               |              |

What did the debouncer fix, and roughly how many extra counts was bounce producing per press?

---

---

Why is the two-flip-flop synchronizer needed even though the debouncer already filters the input?

---

---

## Part 6 — ROM Addressed by the Counter

---

*List the 16 values you stored.*

| Addr  | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  |
|-------|---|---|----|----|----|----|----|----|
| Value |   |   |    |    |    |    |    |    |
| Addr  | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
| Value |   |   |    |    |    |    |    |    |

Did the LEDs display your pattern rather than a binary count? \_\_\_\_\_

| Resource               | Used |
|------------------------|------|
| LUTs                   |      |
| Block RAM (BRAM tiles) |      |

Was the ROM built from LUTs or block RAM? \_\_\_\_\_

## Part 7 — RAM Write and Read Back

---

| Address | Value written | Value read back | Match? |
|---------|---------------|-----------------|--------|
|         |               |                 |        |
|         |               |                 |        |
|         |               |                 |        |
|         |               |                 |        |
|         |               |                 |        |

## Part 8 — Forcing Block RAM

---

| Implementation          | LUTs used | BRAM tiles used |
|-------------------------|-----------|-----------------|
| Default (tool's choice) |           |                 |
| ram_style = "block"     |           |                 |

Which is the better fit for a 16×4 memory, and why did the tool choose what it did by default?

---

---

## Discussion and Conclusions

*Address at least: why you divide a clock with an enable rather than by clocking logic from a divided signal, what simulation could not have shown you about the button, and when a design should use block RAM instead of distributed RAM.*

[illegible]